



Coding Agents in Noteable

Staff and Student Guidance

Academic Year 2026–27

Noteable now give students access to Codex, Claude and OpenCode coding agents.

These tools are available from the **command line** in all Noteable notebook environments and can support AI-assisted coding, debugging, explanation, notebook development and exploratory programming.

Noteable provides a safe and managed space for students and staff to experiment with AI-assisted and agentic programming. Behind the scenes, Noteable sessions run in containerised notebook environments, providing a strongly sandboxed workspace for code development and testing.

Note that while our Jupyter and RStudio environments are updated outside of teaching time, coding agents will be updated more frequently.

We recognise that requirements around AI use vary by course, cohort and assessment context. Noteable therefore provides flexibility for courses that wish to make use of these features.

For lecturers that do not wish to make coding agents available for their course/s, these can be disabled at institutional or course level.

Please test that the relevant AI tools for your course are available, whether Terminal-based access to Codex works as intended, and confirm that the available options support your intended teaching or workflow use cases.

What is a coding agent?

A **coding agent** is an AI-powered tool designed to help with software development tasks.

A coding agent is different from a general chatbot or standard large language model interface and it can often work more directly with files, code and the command line.

A coding agent may be able to:

- read files in a workspace;
- suggest and make code changes;
- edit or create files;
- explain code
- help debug errors;

- generate tests;
- help with notebooks and scripts;
- support multi-step programming tasks.

A general-purpose LLM usually responds to prompts in a chat interface. A coding agent can take actions in the user's workspace.

Coding agents available in Noteable

The Noteable Summer 2026 update includes access to:

- **Codex**
- **Claude**
- **OpenCode**

These are available from the terminal/command line in Noteable environments.

Noteable provides access to the coding-agent tools. Users will need to authenticate with, or provide an API key for, the relevant AI provider. Access requirements, available models, usage limits and any associated costs vary between coding agents and providers.

Any charges incurred through an AI provider are payable directly to that provider and are not included in a Noteable subscription.

Before using a coding agent in teaching, staff should confirm the required account, authentication and API arrangements, and test the intended workflow in the relevant Noteable environment.

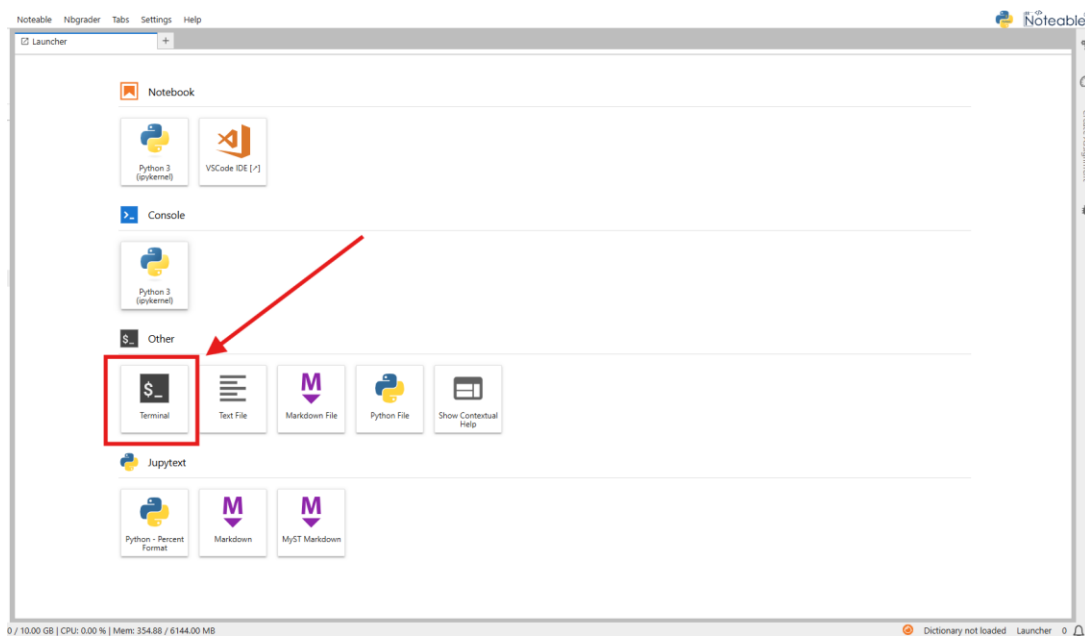
The video below goes over the set up process and getting started with coding agents using an example API key from EDINA's ELM[®] service.*

*See bottom of document for information about ELM.

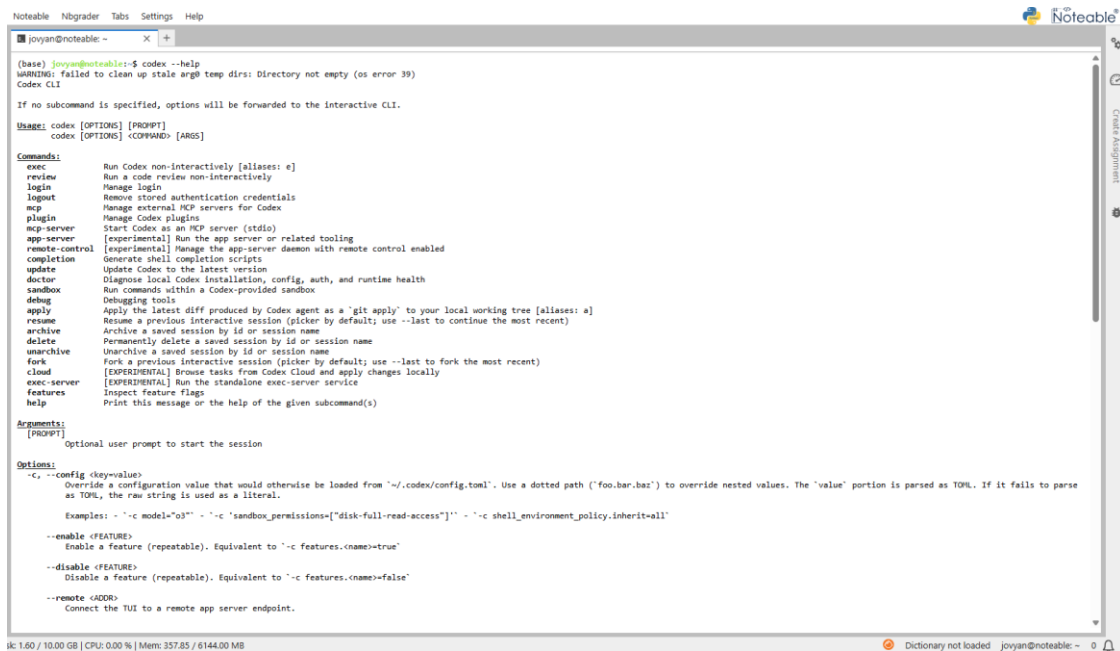


How to access coding agents in JupyterLab notebook servers:

1. Launch a Terminal



2. You can access coding agents directly through the Terminal, get started using `<codingagent> --help`



```
Noteable Nbgrader Tabs Settings Help
jovyan@noteable: ~
(base) jovyan@noteable:~$ codex --help
WARNING: failed to clean up stale argb temp dirs: Directory not empty (os error 39)
Codex CLI

If no subcommand is specified, options will be forwarded to the interactive CLI.

Usage: codex [OPTIONS] [PROMPT]
       codex [OPTIONS] <COMMAND> [ARGS]

Commands:
  exec          Run Codex non-interactively [aliases: e]
  review        Run a code review non-interactively
  login         Manage login
  logout        Remove stored authentication credentials
  mcp           Manage external MCP servers for Codex
  plugin        Manage Codex plugins
  mcp-server    Start Codex as an MCP server (stdio)
  app-server    [experimental] Run the app server or related tooling
  remote-control [experimental] Manage the app-server daemon with remote control enabled
  completion    Generate shell completion scripts
  update        Update Codex to the latest version
  doctor        Diagnose local Codex installation, config, auth, and runtime health
  sandbox       Run commands within a Codex-provided sandbox
  debug         Debugging tools
  apply         Apply the latest diff produced by Codex agent as a 'git apply' to your local working tree [aliases: a]
  resume        Resume a previous interactive session (picker by default; use --last to continue the most recent)
  archive       Archive a saved session by id or session name
  delete        Permanently delete a saved session by id or session name
  unarchive      Unarchive a saved session by id or session name
  fork          Fork a previous interactive session (picker by default; use --last to fork the most recent)
  cloud         [EXPERIMENTAL] Browse tasks from Codex Cloud and apply changes locally
  exec-server   [EXPERIMENTAL] Run the standalone exec-server service
  features      Inspect feature flags
  help          Print this message or the help of the given subcommand(s)

Arguments:
  [PROMPT]
    Optional user prompt to start the session

Options:
  -c, --config <key=value>
    Override a configuration value that would otherwise be loaded from '~/.codex/config.toml'. Use a dotted path ('foo.bar.baz') to override nested values. The 'value' portion is parsed as TOML. If it fails to parse as TOML, the raw string is used as a literal.

    Examples: - '-c model="o3"' - '-c 'sandbox_permissions=["disk-full-read-access"]' - '-c shell_environment_policy.inherit=all'

  --enable <FEATURE>
    Enable a feature (repeatable). Equivalent to '-c features.<name>=true'

  --disable <FEATURE>
    Disable a feature (repeatable). Equivalent to '-c features.<name>=false'

  --remote <ADDR>
    Connect the TUI to a remote app server endpoint.
```

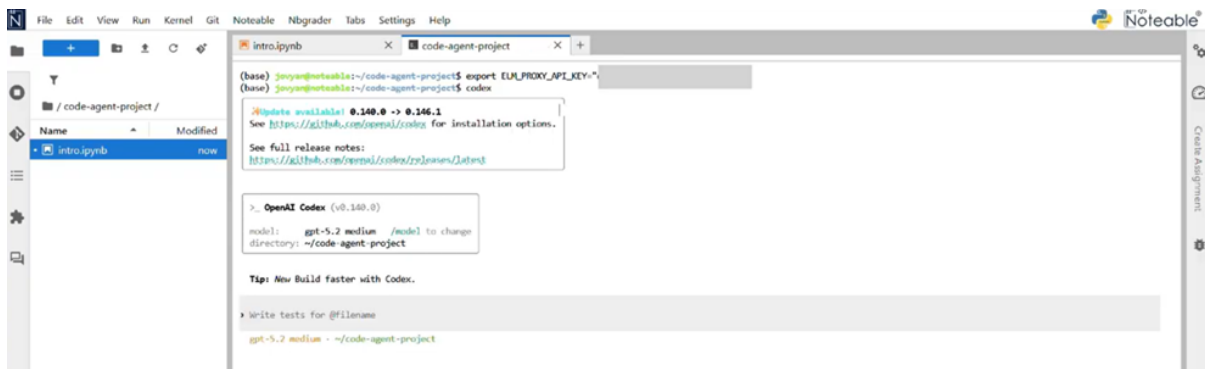
Codex

in the Terminal in Noteable JupyterLab environments

You can then write prompts and run relevant commands, for example:

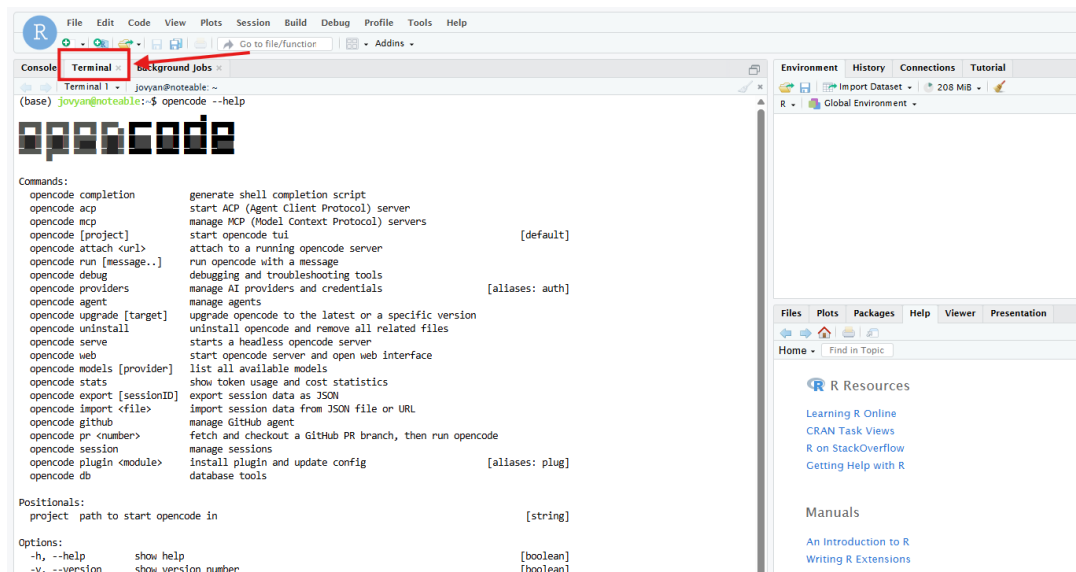
`codex` starts the codex interactive session, while

`codex --help` displays available commands and usage options.



In RStudio:

1. Launch your Noteable RStudio session.
2. Open the **Terminal** pane (found in the upper-left quadrant; if not visible, go to *Tools > Global Options > Terminal* or click the "Terminal" tab).
3. Run the agent commands directly (e.g., `opencode --help`, `claude --help`, or `codex --help`).
4. *Note:* These agents operate at the system level within the container, allowing them to assist with R script development, package management, and file operations directly from the RStudio terminal.



Coding agents may be updated more frequently than the main notebook environments. Notebook environments are usually updated outside teaching time, whereas coding agents may receive more frequent updates as provider tools and integrations change.

Why Noteable is a suitable place to use coding agents

Noteable is an excellent safe space for using coding agents because it provides a controlled teaching and learning environment.

Key features include:

- containerised notebook sessions;
- sandboxed user environments;
- consistent course environments;
- support for notebook-based teaching;
- options to enable or disable tools at institutional or course level.

This allows students to experiment with AI and agentic programming in a safer environment than working directly on a personal machine or unmanaged cloud system.

Testing checklist for staff

Before using coding agents in teaching, staff should test that:

- the required coding agents are available in the selected Noteable environment;
- terminal-based access works as intended;

- authentication works as expected;
- Codex, Claude and/or OpenCode support the intended workflow;
- students can access the relevant tools;
- the course position on AI use is clearly communicated;
- any required logging, declaration or acknowledgement process is in place.

ELM as an additional AI service

ELM® is EDINA's generative AI service: <https://elm.edina.ac.uk/>

For institutions interested in adding ELM® to their wider digital learning provision, it can support access to approved AI models and compatible coding-agent workflows through ELM® API keys.

To discuss ELM® for your institution, please contact EDINA.

Support and issue reporting

When reporting coding agent issues, please include:

- your university username;
- Course area and notebook environment used;
- the coding agent used and the command or workflow attempted;
- screenshots or error messages.

Please report issues via the Noteable contact form: <https://noteable.edina.ac.uk/contact-us/>